

Reporte de Práctica: Integración y Enriquecimiento de Registros con Promtail y Loki

Nombre del Alumno: Saul Noe Perez Aguilera

Matrícula: 2023371155

Materia: Seguridad en el Desarrollo de Aplicaciones

Práctica: SEGG-U2-P4H-2

Fecha: 14 de Julio de 2026

1. Objetivo

Configurar Promtail para recolectar y enriquecer los archivos de registro generados por VulnerableApp, enviarlos a Loki y validar que los mismos eventos puedan consultarse tanto desde Seq como desde Grafana, sin modificar la lógica de negocio de la aplicación.

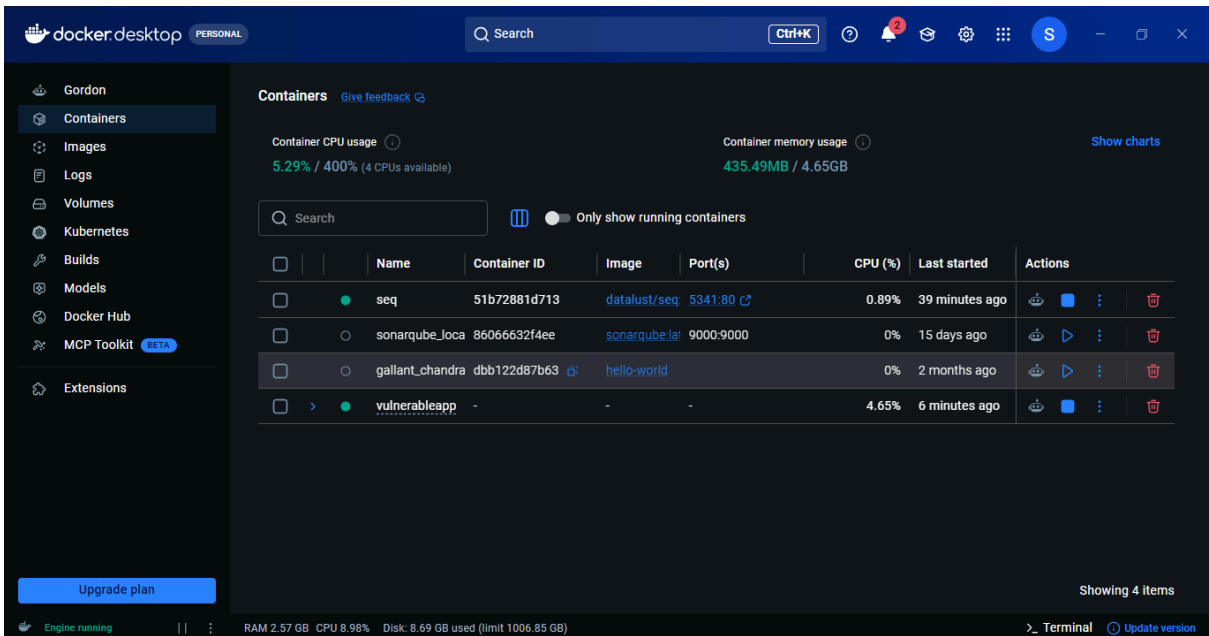
```
Problems  Output  Debug Console  Terminal  Ports
PS D:\Documentos\ing_2doCuatri\Seguridad en desarrollo de aplicaciones\Practica2\VulnerableApp> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
052eb3e65902   grafana/promtail:latest             "/usr/bin/promtail -..." 2 days ago    Up 11 minutes
3d6f9ef6e5d9   grafana/grafana:latest              "/run.sh"                2 days ago    Up 11 minutes    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
0eff39640c0d   grafana/loki:latest                 "/usr/bin/loki -conf..." 2 days ago    Up 11 minutes    0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
51b72881d713   datalust/seq:latest                 "/bin/seqentry"           8 days ago    Up 8 minutes     0.0.0.0:5341->80/tcp, [::]:5341->80/tcp
PS D:\Documentos\ing_2doCuatri\Seguridad en desarrollo de aplicaciones\Practica2\VulnerableApp>
```

2. Desarrollo de la Práctica

Paso 1. Verificar la infraestructura existente

Se comprobó que los contenedores de Grafana, Loki y Promtail se encontraran en ejecución y que VulnerableApp estuviese generando registros tanto en consola como en Seq.

Evidencia 1: Captura de docker ps y de Seq mostrando eventos.



Paso 2. Análisis del archivo de configuración de Promtail (promtail-config.yml)

A continuación, se describe la finalidad de cada sección principal del archivo:

Sección	Función Principal
---------	-------------------

server	Configura el servidor interno (HTTP/gRPC) de Promtail. Define los puertos de escucha para exponer métricas de salud y herramientas de depuración.
clients	Define el destino al cual Promtail enviará los registros recolectados. En este caso, apunta a la API de ingesta de Loki (http://loki:3100/loki/api/v1/push).
positions	Especifica un archivo local (/tmp/positions.yaml) donde Promtail guarda la ubicación (posición en bytes) hasta donde ha leído cada archivo de log, evitando duplicar registros si el contenedor se reinicia.
scrape_configs	Sección principal donde se definen los trabajos (jobs) de recolección de logs, es decir, de qué origen recolectar y cómo procesarlos.
static_configs	Define los archivos de origen a leer (mediante la etiqueta __path__) y asigna las etiquetas (labels) estáticas iniciales para ese conjunto de logs.
labels	Pares clave-valor que se adjuntan a cada línea de log. Constituyen la base de la indexación en Loki y permiten clasificar y filtrar registros eficientemente.

Paso 3. Configuración del monitoreo de múltiples archivos

Para que Promtail pueda leer múltiples archivos de registro de VulnerableApp, se utilizó un patrón de búsqueda (glob) dentro de la etiqueta __path__:

yaml

```
__path__: /app/Logs/{log-*.txt,security-*.txt}
```

¿Por qué Promtail requiere conocer la ubicación física de los archivos? Promtail funciona realizando tailing continuamente directamente sobre el sistema de archivos del SO. Necesita la ruta física para abrir los descriptores de archivo, calcular los desplazamientos (offsets) y detectar rotaciones de archivos (cuando se crea un archivo nuevo de logs). Por ello, se monta el volumen local de los logs hacia dentro del contenedor en Docker Compose.

Paso 4. Enriquecimiento de registros mediante Labels

Se configuraron las siguientes etiquetas para distinguir los registros:

Application: Eventos generales del ciclo de vida de la aplicación.

Security: Eventos críticos como intentos de acceso, fallos de login y logout.

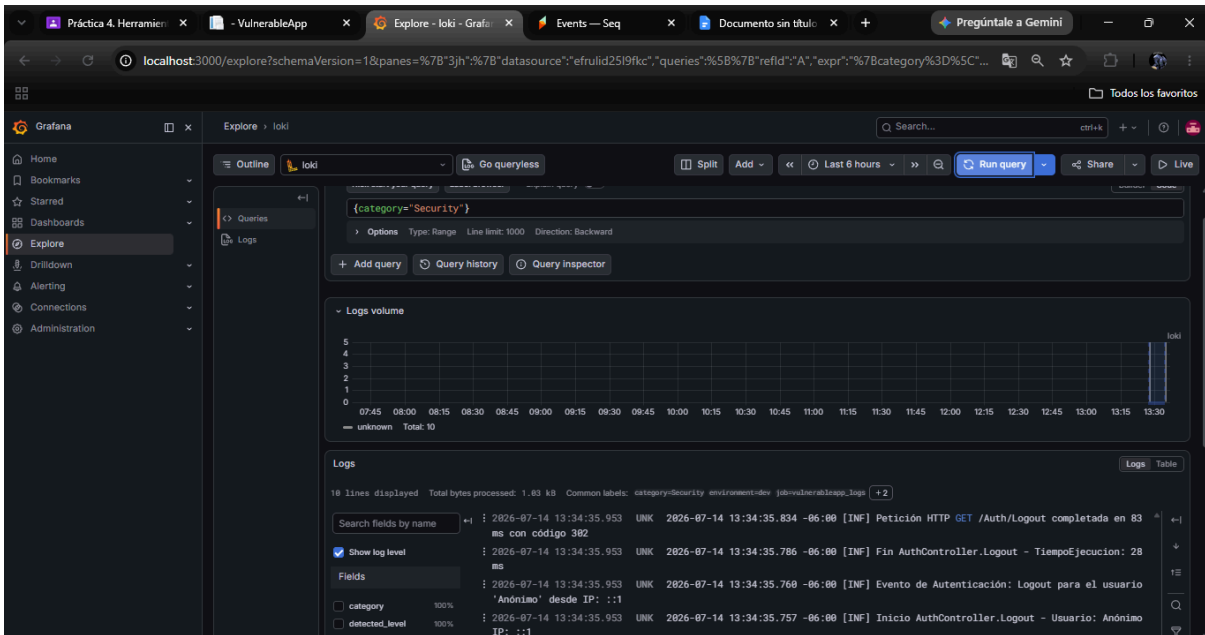
Audit: Operaciones importantes como adición de comentarios, búsquedas y uso de APIs.

Ventajas frente a una búsqueda textual: Las labels son indexadas de forma independiente por Loki. Al realizar un filtrado como {category="Security"}, el motor no necesita escanear línea por línea todos los gigabytes de logs usando costosas expresiones regulares para encontrar palabras de seguridad. Solo busca dentro del pequeño subconjunto pre-etiquetado en Promtail, haciendo las consultas increíblemente más rápidas, escalables y estructuradas.

Pasos 5 al 7. Aplicación, Generación de Actividad y Validación en Grafana

Se reinició el contenedor de Promtail para aplicar la configuración y se interactuó con VulnerableApp. Los resultados filtrados por las etiquetas creadas se observan a continuación en Grafana:

Evidencia 2: Captura de Grafana mostrando consultas por labels.



D

3. Análisis Comparativo: Seq vs Grafana + Loki (Paso 8)

Característica	Seq	Grafana + Loki
Búsqueda textual	Excelente. Orientada a propiedades estructuradas generadas directamente desde el código (Serilog).	Muy potente pero basada en texto plano usando expresiones regulares de LogQL (~ "error").
Filtros por labels	Los filtros se basan en las propiedades nativas del log (ej. @Level, MachineName).	Se basa en labels indexadas inyectadas por el recolector (Promtail). Ideal para segmentar y aislar aplicaciones.
Dashboards	Posee paneles analíticos predefinidos útiles, pero con personalización visual limitada.	Excepcional. Permite crear dashboards altamente personalizables cruzando Logs y Métricas.
Consultas avanzadas	Usa sintaxis similar a SQL nativa para agrupar, filtrar y contar eventos estructurados.	Usa LogQL, que permite extraer métricas desde logs y graficar tasas (rates) en tiempo real.

Escalabilidad	Moderada. Requiere licenciamiento de pago en despliegues distribuidos grandes (clúster).	Altamente escalable, diseñado cloud-native para ingesta masiva (S3, GCS) y entornos de Kubernetes.
Uso principal	Análisis de logs detallado para desarrolladores (.NET) buscando errores y trazas complejas.	Observabilidad centralizada de infraestructura (DevOps/SRE) uniendo múltiples microservicios.

4. Pregunta de Reflexión

¿Qué ocurriría si Promtail no almacenara la posición del último registro leído en `positions.yaml` y cómo afectaría la recolección de eventos? Si no se almacenara esta posición, Promtail perdería el rastro de qué líneas de texto ya ha procesado. Al reiniciarse el contenedor o el servicio, Promtail abriría los archivos de registro y comenzaría a leerlos nuevamente desde el principio (desde la línea 1). Efecto: Esto afectaría gravemente la recolección, provocando que todos los eventos históricos se dupliquen repetidamente en Loki tras cada reinicio, saturando el almacenamiento y alterando falsamente las gráficas y métricas visualizadas en los dashboards.

5. Reto: Configuración de Labels Adicionales

Se configuraron las siguientes labels en el archivo `promtail-config.yml` para aislar los flujos de logs y resolver el reto:

`environment=dev` (Etiqueta estática para todos los logs).

`module=authentication` (Etiqueta dinámica mediante pipeline stages para registros de login y accesos).

`module=orders` (Etiqueta dinámica para operaciones referentes a pedidos/orders).

Procedimiento realizado en Grafana: Para filtrar únicamente los registros del módulo de Autenticación, se accedió a la vista de Explore, se seleccionó la fuente de datos Loki, y se ejecutó la siguiente consulta LogQL:

```
logql
```

```
{module="authentication"}
```

O de forma compuesta:

```
logql
```

```
{environment="dev", module="authentication"}
```

Evidencia 3: Archivo `promtail-config.yml` actualizado y captura del reto en Grafana.

Archivo `promtail-config.yml` Final Modificado:

```
yaml
```

```
server:
```

http_listen_port: 9080

grpc_listen_port: 0

positions:

filename: /tmp/positions.yaml

clients:

- url: http://loki:3100/loki/api/v1/push

scrape_configs:

- job_name: vulnerableapp

static_configs:

- targets:

- localhost

labels:

job: vulnerableapp_logs

__path__: /app/Logs/{log-*.txt,security-*.txt}

category: Application

environment: dev

pipeline_stages:

- match:

selector: '{job="vulnerableapp_logs"} |~ "(?i)(Autenticación|Fallo de login|Intento de acceso|Logout)'"

stages:

- labels:

category: "Security"

module: "authentication"

- match:

selector: '{job="vulnerableapp_logs"} |~ "(?i)(AddComment|ApiController|Search|Usuario:)" !~ "(?i)(Autenticación|Fallo de login|Intento de acceso|Logout)'"

stages:

- labels:

category: "Audit"

- match:

selector: '{job="vulnerableapp_logs"}' |~ "(?i)(order|pedido)"

stages:

- labels:

module: "orders"

